

Universidad Nacional de Río Cuarto

Facultad de Cs Exactas

Carrera de Licenciatura en Ciencias de la Computación

Taller de Diseño de Software

Profesor: Francisco Bavera (Pancho)

Integrantes:

Gonzalo Trepin

Esteban Galucci

Florencia Hernández

Informe de Proyecto de Compiladores

7 de noviembre de 2025

Índice

1. Análisis Léxico	3
1.1. División del trabajo	3
1.2. Decisiones de diseño y asunciones	3
1.3. Diseño y decisiones clave	3
1.4. Detalles interesantes	3
2. Análisis Sintáctico (Parser)	4
2.1. Decisiones de diseño y asunciones	4
2.2. Diseño y decisiones clave	4
2.3. Detalles de implementación interesantes	4
2.4. Problemas conocidos	4
3. Construcción del Árbol de Sintaxis Abstracta (AST)	5
3.1. Decisiones de diseño y asunciones	5
3.2. Diseño y decisiones clave	5
3.3. Detalles de implementación interesantes	5
4. Tabla de Símbolos y Análisis Semántico	6
4.1. Diseño y decisiones clave	6
4.2. Estructura de información	6
4.3. Detalles de implementación interesantes	7
5. Chequeo de Tipos (Typecheck)	7
5.1. Recorrido del AST	7
5.2. Chequeos realizados	7
5.3. Manejo de errores	8
5.4. Uso de <code>eval_type</code>	9
5.5. Ventajas del diseño	9
5.6. Limitaciones y mejoras	9
6. Generación de Código Intermedio (TAC)	9
6.1. Diseño y decisiones clave	9
6.2. Estructura y almacenamiento	9
6.3. Generación de TAC	9
7. Generación de Código Ensamblador	10
7.1. Diseño y decisiones clave	10
7.2. Manejo de Variables y Temporales	10
7.3. Traducción de Instrucciones TAC	10
7.4. Problemas Conocidos y Detalles de Implementación	11
8. Optimización	12
8.1. Objetivo	12
8.2. Diseño y decisiones clave	12
8.3. Detalles de implementación interesantes	13
8.4. Problemas encontrados	13
8.5. Resultados y mejoras observadas	13
8.6. Propagación de Constantes	13
8.7. Eliminación de Código Muerto (Dead Code Elimination)	15

8.8. Limitaciones y posibles mejoras	19
--	----

1. Análisis Léxico

1.1. División del trabajo

En esta etapa trabajamos en conjunto. La principal tarea consistió en reconocer los tokens del lenguaje (palabras reservadas, identificadores, literales, operadores, símbolos y comentarios) y plasmarlos en el archivo `flex.l`.

1.2. Decisiones de diseño y asunciones

El analizador léxico fue implementado con **Flex**, generando el archivo `lex.yy.c`. Se nos proporcionó un listado de tokens y reglas exactas. Sin embargo, la definición de ciertos elementos, como los identificadores, la dedujimos directamente de la gramática.

Por decisión de diseño, cada token reconocido se imprime en un archivo de salida (`lexout`) con fines de depuración, lo cual permitió seguir el flujo del análisis léxico de manera clara.

1.3. Diseño y decisiones clave

- Los identificadores se reconocen mediante la expresión:

```
{letra}({letra}|{digito}|_)*
```

asegurando que comiencen con una letra y luego puedan incluir letras, dígitos o guiones bajos.

- Los literales enteros se reconocen mediante:

```
{digito}+
```

convirtiendo el texto a entero con `atoi`.

- Los literales booleanos se mapean directamente a los valores `true` y `false`.
- Los comentarios de una línea se manejan con `//.*`, mientras que los comentarios multilínea se manejan con:

```
"/*"([~*]|\\n|\\*+[^*/]))*"*/
```

1.4. Detalles interesantes

Un detalle particular fue la necesidad de manejar comentarios multilínea. Si bien logramos implementar una expresión regular que funciona en la mayoría de los casos, aún presenta problemas en situaciones de **comentarios anidados**, esto todavía está pendiente de solucionar, aunque no lo consideramos algo crítico, por lo que puede esperar.

2. Análisis Sintáctico (Parser)

2.1. Decisiones de diseño y asunciones

La gramática proporcionada contenía **ambigüedades** y algunas expresiones regulares. Por lo tanto, el primer paso fue **desambiguar la gramática y eliminar las expresiones regulares**, traduciendo estas reglas a una forma compatible con Bison sin perder el significado del lenguaje.

Para manejar operadores, se definieron precedencias usando `%left` y `%prec` para diferenciar operadores unarios y binarios.

2.2. Diseño y decisiones clave

- **Asociación de operadores:** Problemas de ambigüedad se resolvieron usando las directivas `%left` y `%right` de Bison.
- **Operador unario vs binario:** El operador `-` podía aparecer tanto como unario (negación) o binario (resta). Esto se resolvió usando la directiva `%prec UMINUS` en la regla correspondiente, diferenciando correctamente los contextos.
- **Conflictos shift/reduce por derivaciones vacías:** La gramática original permitía que ciertas reglas, como `var_decls` y `method_decls`, derivaran a λ (cadena vacía), provocando conflictos shift/reduce. Por ejemplo, la regla original:

```
program : PROGRAM '{' var_decls method_decls '}' ;
```

se transformó en:

```
program
  : PROGRAM '{' var_decls method_decls '}'
  | PROGRAM '{' var_decls '}'
  | PROGRAM '{' method_decls '}'
  | PROGRAM '{' '}'
  ;
```

Esto elimino la necesidad de tener la regla vacia en `var_decls` y `method_decls` , cubriendo todos los casos posibles y eliminando los conflictos.

2.3. Detalles de implementación interesantes

- Uso de `%prec UMINUS` para diferenciar operadores unarios y binarios.
- Expansión de reglas para eliminar derivaciones vacías y prevenir conflictos shift/reduce.

2.4. Problemas conocidos

Hasta el momento, el parser reconoce correctamente todos los constructos del lenguaje. No se han detectado conflictos sintácticos pendientes.

3. Construcción del Árbol de Sintaxis Abstracta (AST)

Para la construcción del AST, reutilizamos la definición que habíamos hecho en el pre-proyecto extendiendo la estructura existente para adaptarla al lenguaje completo.

3.1. Decisiones de diseño y asunciones

Se agregaron nuevos tipos de nodos que no estaban presentes en el pre-proyecto, ya que el lenguaje actual los requería: `NODE_UNOP`, `NODE_WHILE`, `NODE_IF`, `NODE_PROG`, `NODE_CALL` y `NODE_PARAM`.

Cada nodo del AST contiene:

- **Tipo de nodo (NodeType):** indica la categoría sintáctica (por ejemplo, entero, booleano, asignación, binop, función, etc.).
- **Campo info:** almacena información relevante, como nombre de variable o función, valor de literales enteros (`ival`) o booleanos (`bval`), y tipo de evaluación (`eval_type`). El `eval_type` permite determinar el tipo de retorno de funciones o de expresiones, y será útil en el futuro chequeo de tipos.

Actualmente, todos los nodos reservan espacio para la misma información, aunque muchos nodos no lo usan. Esto podría optimizarse en el futuro definiendo distintos tipos de `info` según el nodo.

3.2. Diseño y decisiones clave

- Cada nodo puede tener un **hijo izquierdo**, un **hijo derecho** y un **nodo next** que apunta al hermano derecho. Esto permite representar listas de elementos (por ejemplo, listas de sentencias, parámetros o argumentos) de manera sencilla.
- Se construye el AST de manera incremental durante el parseo, usando las acciones de Bison para crear nodos y conectarlos.
- Para nodos de funciones y variables, se inicializa `eval_type` según el tipo declarado (`integer`, `bool`, `void`).
- Para operadores binarios (`BINOP`), el tipo se determinará una vez que se conozcan los tipos de los hijos. Esto permitirá validar operaciones y asignar correctamente `eval_type` al nodo padre.
- Se mantuvo una estructura de árbol flexible que rompe ligeramente el concepto de árbol binario, pero simplifica el manejo de listas y recorridos.

3.3. Detalles de implementación interesantes

- El nodo `next` facilita recorrer listas de nodos, por ejemplo para iterar sobre parámetros de funciones o sentencias dentro de un bloque.
- Cada acción en Bison llama a `make_node` para crear nodos con la información correspondiente y conectarlos según la estructura del AST.
- Literales enteros y booleanos se crean directamente con su valor y tipo asociado (`TYPE_INT`, `TYPE_BOOL`).

4. Tabla de Símbolos y Análisis Semántico

En esta etapa, se realizó la construcción de la tabla de símbolos y el etiquetado del AST, para la tabla de símbolos se implementó la estructura y se construye recorriendo el árbol.

4.1. Diseño y decisiones clave

- La tabla de símbolos (SymTab) se implementa como una estructura anidada con puntero a `parent` y un nivel de `scope`. Menor nivel indica mayor jerarquía, de modo que el nivel 0 corresponde al scope global.
- Cada vez que se encuentra un nodo `BLOCK` en el AST, se abre un nuevo scope en la tabla de símbolos. cuando este bloque termina. se "desapila" toda la información recolectada en ese nivel. esto tiene la desventaja que la tabla se construye y se destruye en el mismo momento de crearse. pero para la utilidad que le damos este funcionamiento es óptimo. los niveles nos permiten manejar variables locales y globales de manera jerárquica, siguiendo el concepto de pila de scopes.
- Cada nodo `ID` o `CALL` en el AST se ".etiqueta" (*labeling*) con su información correspondiente en la tabla de símbolos ya que ambas estructuras almacenan el mismo struct `INFO`. Es decir, el campo `info` del nodo apunta a la misma estructura `Info` que se encuentra en la tabla.
- Gracias a la jerarquía de niveles, siempre se toma la última declaración válida del símbolo en el scope más cercano. Por ejemplo:

```
integer x = 7;
bool test() {
    integer x = 6;
    return x == 7;
}
```

Al evaluar `x == 7` dentro de `test()`, el `ID` se liga al `x` local cuyo valor es 6 ya que la otra declaración de `x` se encuentra un nivel más atrás.

- Se tomó la decisión de que los parámetros de una función pertenezcan al mismo nivel que el bloque de la función. Para esto se distingue en `Info` si un nodo representa una función (ya que por el funcionamiento previo los parámetros quedaban al mismo nivel que la declaración de la función y no al nivel de su bloque), se almacena una lista de parámetros y el nivel de scope inicial es -1 hasta etiquetar el AST.

4.2. Estructura de información

La información almacenada tanto en la tabla de símbolos como en los nodos del AST está definida por la siguiente estructura:

```
typedef struct Params{
    char* param_name;
    TypeInfo param_type;
    struct Params *next;
```

```

} Params;

typedef struct Info {
    char* name;
    int ival;
    int bval;
    int scope;
    char* op;
    TypeInfo eval_type;
    int is_function;
    Params *params;
} Info;

```

como se ve el struct INFO sufrio cambios, los cuales tambien afectan al ast. si bien el funcionamiento no se ve afectado. a nivel arquitectural es una decision bastante pobre ya que se continua gastando memoria en informacion inutil. la mayoria de los campos se usan solo en casos especificos.

4.3. Detalles de implementación interesantes

- La tabla funciona conceptualmente como una pila: los scopes anidados permiten encontrar primero la declaración más cercana de una variable o función.
- El etiquetado del AST vincula cada nodo ID o CALL a la información de la tabla de símbolos correspondiente, lo que facilita el chequeo de tipos y otras verificaciones semánticas posteriores.
- Se implementan verificaciones clásicas de una tabla de símbolos: no se permite re-definir variables o funciones en el mismo scope, y se detecta uso de variables no declaradas.

5. Chequeo de Tipos (Typecheck)

Una vez creada la tabla de símbolos y etiquetado completamente el AST, se realiza el *typecheck* recorriendo el árbol y propagando los tipos hacia arriba. Cada nodo del AST contiene un campo `info->eval_type` que almacena su tipo, lo cual permite verificar la consistencia de tipos de manera eficiente.

5.1. Recorrido del AST

El *typecheck* se realiza mediante un recorrido completo del AST, siguiendo los nodos hijos (`left`, `right`) y hermanos (`next`). Según el tipo de nodo, se aplican las reglas de tipo correspondientes.

5.2. Chequeos realizados

- Operaciones binarias (NODE_BINOP):
 - Los operandos deben tener el mismo tipo.
 - Operadores de comparación (`==`, `<`, `>`) producen tipo `bool`.

- Operadores aritméticos (+, -, *, /, %) requieren operandos enteros y producen tipo `int`.
- Operadores lógicos (&&, ||) requieren operandos booleanos y producen tipo `bool`.
- **Operaciones unarias (NODE_UNOP):**
 - El operador - requiere un entero.
 - El operador ! requiere un booleano.
 - El tipo del nodo se hereda del hijo.
- **Asignaciones (NODE_ASSIGN):**
 - El tipo de la variable debe coincidir con el tipo de la expresión asignada.
- **Declaraciones de variables (NODE_VAR_DECL):**
 - Se chequea el tipo de la expresión de inicialización.
- **Retornos (NODE_RETURN):**
 - Si no hay expresión se espera `void`.
 - Si hay expresión, se compara con el tipo de la función (`func_type`).
- **Funciones (NODE_FUNCTION):**
 - Se guarda el tipo de la función en `func_type`.
 - Se chequean los tipos de parámetros y del bloque de la función.
- **Bloques, condicionales y bucles (NODE_BLOCK, NODE_IF, NODE_WHILE):**
 - Se asegura que las expresiones de condición sean booleanas.
- **Llamadas a funciones (NODE_CALL):**
 - Se comparan los tipos de parámetros reales con los formales.
 - Se detecta exceso o falta de parámetros.

5.3. Manejo de errores

Cada discrepancia de tipos genera un mensaje de error indicando el tipo esperado y el tipo real. Se utiliza la variable global `type_check_error` para registrar si hubo errores durante el chequeo. El recorrido continúa para detectar múltiples errores antes de detener la ejecución. luego de creer terminado el análisis de tipos, detectamos varios errores mas. por ejemplo para una operacion booleana de `==` la logica que seguiamos en typecheck era simplemente, mirar que el tipo de sus operandos sean iguales. esto a priori deberia funcionar, pero el lenguaje aceptaba cosas del tipo `void f(){} void main(){bool test = f() == f();}` esto carece de sentido. otro error que pudimos encontrar fue que podiamos llamar a funciones no existentes. es decir era valido `integer x = 0; integer y = x();`; esto era por un error en la busqueda en la tabla de simbolos, ya que simplemente buscaba el primer identificador que coincidiera en nombre sin vericar si este es funcion o no. estos errores fueron arreglados.

5.4. Uso de `eval_type`

Cada nodo del AST almacena su tipo en `info->eval_type`, lo que permite propagar el tipo desde los hijos hacia los padres y verificar asignaciones, retornos y llamadas a funciones.

5.5. Ventajas del diseño

- Integración directa con la tabla de símbolos: cada nodo `ID` o `CALL` ya está etiquetado con su `info`.
- Chequeo de tipos consistente con la jerarquía de scopes.
- Permite detectar errores de manera precisa y localizada.

5.6. Limitaciones y mejoras

- Los mensajes de error no incluyen la línea exacta.
- No se implementan coerciones de tipos; todo debe coincidir exactamente.
- Se podría optimizar separando el chequeo de expresiones y declaraciones en funciones independientes.

6. Generación de Código Intermedio (TAC)

6.1. Diseño y decisiones clave

Para la representación intermedia del código, se optó por utilizar Código de Tres Direcciones (TAC - Three Address Code), siguiendo las pautas del proyecto. Esta representación simplifica la traducción posterior a código ensamblador y facilita posibles optimizaciones.

Se definió una estructura `TAC` para representar cada instrucción, conteniendo la operación (`op`), hasta dos argumentos (`arg1`, `arg2`) y un destino (`target`). Las instrucciones TAC se generan recorriendo el AST después del análisis semántico y el chequeo de tipos.

6.2. Estructura y almacenamiento

Inicialmente, la generación de TAC se implementó imprimiendo directamente las instrucciones a medida que se recorrían los nodos del AST. Sin embargo, al comenzar la implementación del generador de código ensamblador, nos dimos cuenta de que necesitábamos acceder a la secuencia completa de instrucciones TAC para realizar la traducción.

Para solucionar esto, se modificó la implementación para almacenar las instrucciones TAC generadas en una lista enlazada. Se utilizan los punteros globales `tac_head` y `tac_tail` para manejar la lista. La función `emit_tac` se encarga de crear un nuevo nodo TAC y agregarlo al final de esta lista.

6.3. Generación de TAC

La función principal `generate_tac` inicializa los contadores de temporales y etiquetas y llama a `gen_stmt` sobre la raíz del AST. Las funciones `gen_stmt` y `gen_expr` recorren recursivamente el árbol:

- **gen_expr**: Procesa nodos de expresiones (`INT`, `BOOL`, `ID`, `BINOP`, `UNOP`, `CALL`), genera el TAC correspondiente y devuelve el nombre del registro temporal o variable que contiene el resultado. Utiliza `new_temp` para crear nombres únicos para resultados intermedios.
- **gen_stmt**: Procesa nodos de sentencias (`ASSIGN`, `IF`, `WHILE`, `RETURN`, `FUNCTION`, `BLOCK`, `CALL`, `PROG`), llamando a `gen_expr` cuando es necesario y generando el TAC de control de flujo o asignación. Utiliza `new_label` para crear etiquetas únicas para saltos.

Finalmente, `print_tac_list` recorre la lista enlazada y escribe las instrucciones TAC en el archivo de salida.

7. Generación de Código Ensamblador

7.1. Diseño y decisiones clave

El objetivo de esta etapa es traducir la lista de instrucciones TAC (generada en la etapa anterior) a código ensamblador x86-64 para la plataforma Linux, utilizando la sintaxis de ATT (aunque el código generado parece usar más la sintaxis Intel con `'mov dst, src'`) [cite: 802].

La generación se realiza recorriendo la lista enlazada de instrucciones TAC (`tac_head`) y traduciendo cada instrucción a una o más instrucciones ensamblador.

7.2. Manejo de Variables y Temporales

Se decidió mapear todas las variables locales y temporales del TAC a ****ubicaciones en la pila**** de la función actual. Se utiliza una lista enlazada (`VarMap`) gestionada por `var_map_head` para llevar un registro de qué variable corresponde a qué desplazamiento (*offset*) relativo al puntero base (`rbp`).

La función `ensure_offset` busca una variable en el mapa; si no la encuentra, le asigna un nuevo offset negativo respecto a `rbp` (incrementando `current_stack_size`), reserva espacio en la pila (`sub rsp, 8`) y la agrega al mapa. Las funciones auxiliares `load_i32_to_eax`, `load_i32_to_ecx` y `store_eax_to` se usan para mover datos entre la pila y los registros `eax/ecx`.

El mapa de variables y el tamaño de la pila se reinician al principio de cada función (`TAC_DEFUNC`).

7.3. Traducción de Instrucciones TAC

- **Funciones**: `TAC_DEFUNC` genera el prólogo (`push rbp, mov rbp, rsp`) y la etiqueta global. `TAC_ENDFUNC` y `TAC_RETURN` generan el epílogo (`mov rsp, rbp, pop rbp, ret`). El valor de retorno se deja en `eax`.
- **Asignación y Aritmética**: Instrucciones como `TAC_ASSIGN`, `TAC_SUM`, `TAC_MINUS`, `TAC_MUL`, `TAC_DIV`, `TAC_MOD`, `TAC_NEG` se traducen cargando operandos a `eax` y `ecx`, realizando la operación correspondiente (`mov`, `add`, `sub`, `imul`, `idiv`, `neg`) y guardando el resultado de `eax` en la pila. `idiv` requiere preparar `edx` con `cdq`.
- **Comparaciones y Lógica**: `TAC_LESS`, `TAC_GREATER`, `TAC_EQ` usan `cmp` seguido de instrucciones `set<cond>` (`setl`, `setg`, `sete`) y `movzx` para obtener un resultado

booleano (0 o 1) en `eax`. `TAC_AND`, `TAC_OR`, `TAC_NOT` usan comparaciones con 0 (`cmp`, `test`), `setne`, `sete` y operaciones lógicas (`and`, `or`).

- **Control de Flujo:** `TAC_LABEL` genera una etiqueta. `TAC_GOTO` genera un `jmp`. `TAC_IFZ` genera `test eax, eax` seguido de un salto condicional `je` (Jump if Equal/Zero).
- **Llamadas a Función:** `TAC_PARAM` acumula los argumentos en un array temporal `pending_params`. `TAC_CALL` itera sobre los parámetros acumulados y los mueve a los registros correctos según la convención de llamada x86-64 System V ABI (usando `edi`, `esi`, `edx`, etc., para los primeros argumentos enteros, y la pila para los restantes) mediante `move_arg_to_reg`. Luego ejecuta la instrucción `call` y, si es necesario, limpia la pila de argumentos extra (`add rsp, ...`). El resultado de la llamada (en `eax`) se guarda si `target` no es nulo.

7.4. Problemas Conocidos y Detalles de Implementación

- **Necesidad de Almacenar TAC:** Como se mencionó, fue necesario refactorizar la etapa de código intermedio para almacenar el TAC en una lista enlazada, en lugar de solo imprimirlo, para que el generador de ensamblador pudiera procesarlo.
- **Mejorar estructura para almacenar variables:** Nos dimos cuenta que cuando guardábamos variables pisábamos el valor de la anterior, por lo que solo podíamos almacenar un valor a la vez. Para solucionarlo, agregamos una estructura llamada `VarMap` que lleva registro de todas las variables, con sus nombres y el offset, este es la distancia al base pointer en bytes, lo cual nos deja acceder a su dirección para poder utilizarla.
- **Pasaje de parámetros incorrecto:** Descubrimos que estábamos leyendo mal los parámetros, por esa razón todas las funciones usaban el valor 0 por defecto. Por ejemplo, teníamos una test que calculaba el factorial de un número y este siempre devolvía 1, sea cual sea la entrada, debido a que el factorial de 0 es 1. Se nos ocurrió agregar variables globales para controlar, cuando estamos leyendo parámetros y cuantos vamos leyendo. Con estas variables, pudimos controlar correctamente y nos permitió leer correctamente los parámetros. Al solucionar esto surgió el problema de que solo podíamos almacenar en registro seis parámetros, si teníamos más se rompía. Esto fue causado porque usábamos únicamente los seis registros en lugar de pushear a la pila. Decidimos calcular dinámicamente el offset en base a la cantidad de parámetros para almacenar más que la cantidad de registros. Ahora funciona correctamente con cualquier cantidad.
- **Gestión de Offsets en Pila:** La traducción de nombres de variables y temporales a offsets relativos a `rbp` es crucial. La función `ensure_offset` maneja esto dinámicamente, pero requiere cuidado para asegurar que cada variable tenga un offset único dentro de su ámbito (actualmente, el ámbito de la función).
- **Ineficiencias:** El código ensamblador generado realiza muchos accesos a memoria (pila) incluso para operaciones simples que podrían realizarse enteramente con registros, lo cual es ineficiente. Por ejemplo, cargar constantes a registros solo para guardarlas inmediatamente en la pila y luego volver a cargarlas.
- **Convención de Llamada:** Implementar correctamente la convención de llamada (paso de argumentos en registros y pila, limpieza de pila) es complejo. Aunque se

intenta seguir (usando `edi`, `esi`), requiere atención a los detalles, especialmente con más de 6 argumentos o tipos no enteros (no soportados por TDS25).

8. Optimización

8.1. Objetivo

En esta etapa se buscó mejorar la eficiencia del código generado reduciendo operaciones innecesarias y eliminando fragmentos que nunca se ejecutan. Para ello, implementamos una fase de **optimización sobre el Árbol de Sintaxis Abstracta (AST)** antes de la generación del código intermedio.

8.2. Diseño y decisiones clave

La optimización se centra en dos técnicas principales: **propagación de constantes** y **eliminación de código muerto (dead code elimination)**. Ambas se implementaron directamente sobre el AST, antes de generar el Código de Tres Direcciones (TAC). Además agregamos otras técnicas de optimización como **evaluación cortocircuito** y **Peephole**.

- **Propagación de constantes:** Si una expresión puede evaluarse en tiempo de compilación (por ejemplo, `3 + 4`, `true && false`), el nodo se reemplaza por un nuevo nodo literal con el valor resultante. Esto reduce el tamaño del AST y evita generar instrucciones TAC innecesarias.
- **Eliminación de código muerto:** Se recorren los nodos del AST con una función recursiva llamada `dead_code`, que identifica y elimina subárboles que nunca serán ejecutados. Ejemplos típicos:
 - Todo el código que sigue a un `return` dentro de una función se elimina, ya que es inalcanzable.
 - En estructuras `if-else`, si la condición es una constante, se conserva únicamente la rama correspondiente.
 - En bucles `while` con condición constante falsa, se elimina completamente el cuerpo del bucle.
- **Estrategia incremental:** La optimización se aplica de manera recursiva, revisando cada nodo y realizando simplificaciones en profundidad. Cada vez que un nodo se modifica, el recorrido vuelve sobre sus hijos para detectar posibles nuevas oportunidades de optimización.
- **Evaluación por cortocircuito:** Se aplica en las operaciones lógicas durante la propagación de constantes, con el objetivo de evitar evaluaciones innecesarias. Esta técnica determina cuándo el resultado de una expresión puede conocerse únicamente a partir del primer operando, sin necesidad de evaluar el segundo. Por ejemplo, en una operación OR, si el primer operando es `true`, el resultado será `true` sin evaluar el siguiente; mientras que en una operación AND, si el primer operando es `false`, el resultado será `false` automáticamente.
- **Optimización Peephole:** En las operaciones binarias aritméticas se aplican reglas algebraicas básicas para eliminar cálculos redundantes y simplificar expresiones. Esta técnica, conocida como *peephole optimization*, examina pequeñas secuencias

de instrucciones (una “ventana” o *peephole*) para detectar patrones que puedan ser reemplazados por versiones más eficientes. Por ejemplo, expresiones como $x + 0$ o $x * 1$ se simplifican directamente a x , evitando operaciones innecesarias.

8.3. Detalles de implementación interesantes

- La función `dead_code(AST *n)` recorre el árbol en profundidad (*depth-first*) y modifica los punteros de los nodos en tiempo real. Si detecta un nodo `NODE_RETURN`, elimina todo lo que sigue en la secuencia de sentencias (`n->next`).
- En nodos `NODE_IF` y `NODE_WHILE`, cuando la condición es constante, se decide si conservar o eliminar ramas completas, evitando recorrer innecesariamente partes del árbol.
- Durante la propagación de constantes, los nodos de operaciones binarias (`NODE_BINOP`) o unarias (`NODE_UNOP`) se reemplazan por literales evaluados. Por ejemplo, una expresión $3 * 2$ se convierte directamente en un nodo `INT(6)`.
- Para evitar pérdida de memoria, se liberan los nodos sustituidos mediante `free()` y se reasignan punteros al nuevo nodo simplificado.

8.4. Problemas encontrados

Durante las primeras pruebas, al optimizar de forma recursiva los nodos, se presentaron **violaciones de segmento (segmentation faults)**. Esto ocurría principalmente porque, al eliminar un nodo, se accedía a memoria liberada desde su nodo padre o desde un hermano (`n->next`). El problema se resolvió controlando cuidadosamente los punteros antes y después de liberar memoria, y reasignando correctamente el flujo del recorrido.

También surgió un problema al intentar ejecutar varios `scanf` secuenciales en el código generado. Al analizarlo, descubrimos que el offset se calcula de una manera distinta que el que se necesita para `scanf`. Para solucionarlo se agrego una condición para detectar cuando el offset se encuentra desalineado y así cambia el tamaño del stack para evitar estos errores.

8.5. Resultados y mejoras observadas

- Se redujo significativamente la cantidad de instrucciones TAC generadas, especialmente en bloques con constantes o ramas condicionales simples.
- El código ensamblador final es más corto y limpio, sin secciones redundantes ni saltos innecesarios.
- El análisis de flujo y eliminación de código muerto permitió detectar errores lógicos en el propio lenguaje de entrada (por ejemplo, condiciones inalcanzables o retornos prematuros).

8.6. Propagación de Constantes

La **propagación de constantes** es la primer capa de optimización que aplicamos, posterior a realizar el análisis léxico consiste en sustituir las expresiones cuyo valor puede determinarse estáticamente. De este modo, se reducen tanto la complejidad del AST como la cantidad de instrucciones generadas en código máquina.

A continuación se muestra un ejemplo de la transformación aplicada a la instrucción:

```
a = 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10;
```

AST sin optimización

```
ASSIGN
  ID(a) scope = (0)
  BINOP(+)
    BINOP(+)
      BINOP(+)
        BINOP(+)
          BINOP(+)
            BINOP(+)
              BINOP(+)
                BINOP(+)
                  INT(1)
                  INT(2)
                  INT(3)
                  INT(4)
                  INT(5)
                  INT(6)
                  INT(7)
                  INT(8)
                  INT(9)
                  INT(10)
```

El árbol posee una profundidad de 10 niveles anidados de operaciones binarias, lo que implica una mayor carga para el análisis semántico y la generación de código.

AST tras la propagación de constantes

```
ASSIGN
  ID(a) scope = (0)
  INT(55)
```

Luego de aplicar la propagación, la expresión se reemplaza directamente por su valor constante, reduciendo significativamente la profundidad y cantidad de nodos del árbol.

Impacto en el código ensamblador

La diferencia también se observa claramente en el código generado. Sin aplicar la optimización:

```
.text
...
.globl main
```

```

main:
    pushq    %rbp
    movq     %rsp, %rbp
    movl     $0, %eax
    subq     $8, %rsp
    ...
    ; múltiples operaciones de suma
    movq     %rbp, %rsp
    popq     %rbp
    ret

```

Con la optimización aplicada:

```

.text
...
.globl main
main:
    pushq    %rbp
    movq     %rsp, %rbp
    movl     $55, %eax
    subq     $8, %rsp
    movl     %eax, -8(%rbp)
    movl     -8(%rbp), %eax
    subq     $8, %rsp
    movl     %eax, -16(%rbp)
    movq     %rbp, %rsp
    popq     %rbp
    ret

```

Resultados

La simplificación reduce drásticamente el tamaño del árbol (de 19 nodos a 3) y elimina todas las operaciones aritméticas redundantes. Esto no solo acorta el código ensamblador, sino que además disminuye el tiempo de ejecución y facilita futuras optimizaciones como la eliminación de código muerto o la propagación de copias.

8.7. Eliminación de Código Muerto (Dead Code Elimination)

La segunda optimización que aplicamos fue la **Eliminación de Código Muerto** que tiene como objetivo suprimir del AST todas aquellas instrucciones o bloques que nunca se ejecutan, es decir el código muerto.^{Esta} transformación reduce el tamaño del código intermedio y del código ensamblador generado, mejorando la eficiencia global del compilador.

Ejemplo de aplicación

El siguiente programa fue diseñado para incluir instrucciones inalcanzables con el fin de ilustrar el efecto de la optimización:


```

program {
    integer useless(integer x) {
        integer y = 0;
        y = x * 2;
        return y;
        y = y + 100;    // Código muerto
        x = 0;          // Código muerto
    }

    void main() {
        integer a = 0;
        a = 10;
        if (false) then {
            a = a + 1;  // Código muerto
        } else {
            a = a + 2;
        }

        if (true) then {
            a = a * 2;
        } else {
            a = a * 999; // Código muerto
        }

        return;
        a = 9999; // Código muerto
        a = 9998; // Código muerto
        a = 9997; // Código muerto
        a = 9996; // Código muerto
        a = 0;    // Código muerto
    }
}

```

Árbol antes de la optimización

```

PROGRAM
  FUNCTION(useless)
    PARAM(integer, x)
    BLOCK
      VAR_DECL(y)
      ASSIGN
        ID(y) scope = (1)
        INT(0)
      ASSIGN
        ID(y) scope = (1)
        BINOP(*)
        ID(x) scope = (1)
        INT(2)
    RETURN
      ID(y) scope = (1)

```

```

    ASSIGN
        ID(y) scope = (1)
        BINOP(+)
            ID(y) scope = (1)
            INT(100)
    ASSIGN
        ID(x) scope = (1)
        INT(0)
FUNCTION(main)
    BLOCK
        VAR_DECL(a)
        ASSIGN
            ID(a) scope = (1)
            INT(0)
    ASSIGN
        ID(a) scope = (1)
        INT(10)
    IF
        BOOL(FALSE)
        BLOCK
            ASSIGN
                ID(a) scope = (1)
                BINOP(+)
                    ID(a) scope = (1)
                    INT(1)
        BLOCK
            ASSIGN
                ID(a) scope = (1)
                BINOP(+)
                    ID(a) scope = (1)
                    INT(2)
    IF
        BOOL(TRUE)
        BLOCK
            ASSIGN
                ID(a) scope = (1)
                BINOP(*)
                    ID(a) scope = (1)
                    INT(2)
        BLOCK
            ASSIGN
                ID(a) scope = (1)
                BINOP(*)
                    ID(a) scope = (1)
                    INT(999)
    RETURN
    ASSIGN
        ID(a) scope = (1)
        INT(9999)
    ASSIGN

```

```

        ID(a) scope = (1)
        INT(9998)
    ASSIGN
        ID(a) scope = (1)
        INT(9997)
    ASSIGN
        ID(a) scope = (1)
        INT(9996)
    ASSIGN
        ID(a) scope = (1)
        INT(0)

```

Árbol después de la optimización

```

PROGRAM
  FUNCTION(useless)
    PARAM(integer, x)
    BLOCK
      VAR_DECL(y)
      ASSIGN
        ID(y) scope = (1)
        INT(0)
      ASSIGN
        ID(y) scope = (1)
        BINOP(*)
        ID(x) scope = (1)
        INT(2)
      RETURN
        ID(y) scope = (1)
  FUNCTION(main)
    BLOCK
      VAR_DECL(a)
      ASSIGN
        ID(a) scope = (1)
        INT(0)
      ASSIGN
        ID(a) scope = (1)
        INT(10)
      BLOCK
        ASSIGN
          ID(a) scope = (1)
          BINOP(+)
          ID(a) scope = (1)
          INT(2)
      BLOCK
        ASSIGN
          ID(a) scope = (1)
          BINOP(*)
          ID(a) scope = (1)
          INT(2)

```

RETURN

Análisis de resultados

En el árbol optimizado se observa que:

- Se eliminaron las ramas `if (false)` y los bloques `else` de `if (true)`, al ser inalcanzables.
- Se suprimieron todas las asignaciones posteriores a un `return`.
- Se eliminaron las instrucciones sin efecto observable sobre el flujo del programa.

El resultado final es un árbol considerablemente más compacto, que representa únicamente las operaciones efectivamente ejecutables del programa. Esto reduce la carga de procesamiento en las etapas posteriores del compilador y genera código ensamblador más breve y eficiente.

8.8. Limitaciones y posibles mejoras

- Actualmente la optimización se aplica solo sobre el AST. En el futuro, podría extenderse al nivel TAC para detectar redundancias entre instrucciones.